

Industrial Fire Safety System using ESP32 with IoT-Based Worker Evacuation Monitoring

1. Problem Statement

Introduction

Industrial sectors such as manufacturing plants, chemical industries, warehouses, textile mills, oil refineries, and production units are exposed to a high risk of fire accidents due to electrical faults, overheating machinery, combustible materials, and hazardous chemicals. Fire incidents can cause extensive damage to industrial infrastructure, interrupt production processes, and pose a serious threat to human life. One of the major challenges during such emergencies is ensuring the safe and timely evacuation of all workers present inside the industrial premises.

Traditional fire alarm systems are primarily designed to detect fire and activate audible alarms. Although these systems provide an initial warning, they do not monitor the number of workers inside the building or assist emergency responders in identifying whether everyone has been evacuated safely. In most industries, worker attendance during emergencies is verified manually, which is time-consuming and may delay rescue operations.

The advancement of Internet of Things (IoT) technology has enabled industries to implement intelligent monitoring systems capable of collecting, processing, and transmitting real-time data to cloud platforms. By integrating fire detection, worker occupancy monitoring, and cloud connectivity, industrial safety systems can significantly improve emergency response and evacuation management.

Problem Identification

The existing industrial fire safety systems suffer from several limitations:

- They only generate alarm signals without providing worker occupancy information.
- Manual worker counting during emergencies is slow, inaccurate, and unreliable.
- Traditional systems do not support remote monitoring through cloud platforms.
- Emergency responders cannot determine the exact number of workers trapped inside the affected area.
- Fire status and occupancy data are not stored for future analysis and decision-making.
- Conventional systems lack intelligent automation for monitoring industrial safety conditions in real time.

These limitations reduce the efficiency of emergency evacuation and increase the possibility of injuries and loss of life during fire accidents.

Need for the Proposed System

To overcome these limitations, there is a need for an intelligent Industrial Fire Safety System capable of detecting fire accidents, monitoring worker movement, maintaining the real-time count of workers inside the industry, and transmitting this information to a cloud platform for remote monitoring. Such a system enables industries to monitor emergency situations continuously and assists rescue teams in making quick and informed decisions.

Why ESP32 is Used

The ESP32 microcontroller has been selected as the central controller because it provides advanced processing capabilities along with built-in Wi-Fi and Bluetooth connectivity, making it highly suitable for IoT-based industrial applications.

Compared to conventional microcontrollers, the ESP32 offers:

- Built-in Wi-Fi for direct Internet connectivity without additional modules.
- High processing speed (up to 240 MHz) for handling multiple sensors simultaneously.
- Large Flash memory and RAM for efficient program execution.
- Multiple GPIO pins for interfacing with sensors, LCD modules, and alarm devices.
- Low power consumption suitable for embedded applications.
- Easy integration with IoT cloud platforms such as ThingSpeak.

The ESP32 continuously monitors the flame sensor and IR sensors, updates the LCD display, activates the buzzer during emergencies, and uploads worker count and fire status to the ThingSpeak cloud platform for real-time remote monitoring.

Proposed Solution

The proposed Industrial Fire Safety System integrates an ESP32 microcontroller, flame sensor, two infrared sensors, buzzer, and a 16×2 LCD with I2C interface. The system continuously monitors worker entry and exit using IR sensors and maintains the current worker count inside the industrial premises.

Whenever a fire is detected by the flame sensor, the ESP32 immediately activates the emergency buzzer, displays a **"FIRE ALERT"** message on the LCD, and uploads both the worker count and fire status to the ThingSpeak cloud platform through Wi-Fi. This enables factory supervisors and emergency responders to remotely monitor the emergency situation and determine whether all workers have been evacuated safely.

The proposed system combines fire detection, worker monitoring, cloud communication, and emergency alert generation into a single intelligent embedded solution, thereby improving industrial safety and reducing emergency response time.

2. Scope of the Solution

Overview

The proposed Industrial Fire Safety System is designed to improve workplace safety by integrating fire detection, worker occupancy monitoring, and Internet of Things (IoT) technology into a single embedded system. The system provides continuous monitoring of industrial environments and assists emergency responders during fire incidents by maintaining an accurate count of workers present inside the industrial premises.

The solution focuses on early fire detection, automatic emergency alert generation, real-time worker counting, and cloud-based remote monitoring. It provides a cost-effective prototype suitable for demonstrating smart industrial safety concepts and can be further expanded for real-world industrial applications.

Scope of the Proposed System

The proposed system is capable of:

- Detecting fire at an early stage using a flame sensor.
- Monitoring worker entry and exit using two infrared sensors.
- Maintaining the real-time count of workers inside the industrial premises.
- Displaying worker count and system status on a 16×2 LCD display.
- Activating an audible buzzer immediately after fire detection.
- Uploading worker count and fire status to the ThingSpeak IoT cloud platform through ESP32 Wi-Fi.
- Providing remote monitoring of industrial safety conditions using cloud services.
- Assisting emergency responders by indicating the number of workers remaining inside the building.
- Reducing manual intervention during emergency evacuation.
- Demonstrating an IoT-enabled smart industrial safety solution.

Advantages of the Proposed Solution

The proposed system offers several advantages over conventional fire alarm systems:

- Provides both fire detection and worker occupancy monitoring.
- Enables real-time cloud monitoring through IoT.
- Reduces evacuation time during emergencies.
- Helps rescue teams identify workers remaining inside the building.
- Provides continuous monitoring without manual supervision.

- Displays instant system status on an LCD.
- Generates immediate emergency alarms.
- Cost-effective and easy to implement.
- Low power consumption.
- Easily expandable for industrial applications.

Industrial Applications

The proposed system can be implemented in:

- Manufacturing Industries
- Chemical Processing Plants
- Textile Industries
- Warehouses and Storage Facilities
- Pharmaceutical Industries
- Oil and Gas Refineries
- Power Generation Plants
- Educational Institutions
- Hospitals
- Shopping Malls
- Commercial Buildings
- Smart Factories

Limitations

Although the prototype demonstrates the working concept effectively, it has certain limitations:

- The current prototype uses a flame sensor instead of a dedicated smoke detection sensor.
- Worker counting accuracy depends on the proper placement of IR sensors.
- Internet connectivity is required for cloud data transmission.
- The prototype is designed for demonstration purposes and does not include automatic fire suppression mechanisms.
- Environmental conditions may slightly affect sensor performance.

Future Scope

The proposed system can be further enhanced by integrating:

- Smoke and gas sensors (MQ-2/MQ-135)
- Automatic sprinkler control
- GSM/SMS emergency notifications
- Mobile application for remote monitoring
- AI-based fire image detection using cameras
- Firebase or AWS cloud integration
- RFID or Face Recognition for accurate worker identification
- GPS-enabled emergency location tracking
- LoRa communication for long-distance industrial monitoring
- Voice evacuation announcement system
- Battery backup for uninterrupted operation
- Centralized industrial safety dashboard

Conclusion of Scope

The proposed Industrial Fire Safety System demonstrates how IoT and embedded systems can significantly improve industrial safety by combining fire detection, worker counting, cloud communication, and emergency alert mechanisms into a single intelligent platform. The system provides real-time information to both local operators and remote supervisors, enabling faster emergency response and efficient evacuation management. Due to its low cost, scalability, and ease of implementation, the proposed solution serves as an effective prototype for future smart industrial safety applications.

3. Required Components to Develop the Solution

3.1 Hardware Requirements

ESP32 Development Board

Quantity: 1

The ESP32 Development Board serves as the central controller of the proposed Industrial Fire Safety System. It is a high-performance 32-bit microcontroller equipped with built-in Wi-Fi and Bluetooth connectivity, making it suitable for Internet of Things (IoT) applications. The ESP32 continuously monitors the flame sensor and IR sensors, processes the collected data, controls the LCD display and buzzer, and transmits real-time information to the ThingSpeak cloud platform through Wi-Fi.

Specifications

- Operating Voltage: 3.3 V
- Processor: Dual-Core Tensilica LX6
- Clock Speed: Up to 240 MHz
- Flash Memory: 4 MB
- Wi-Fi Standard: IEEE 802.11 b/g/n
- Bluetooth: Classic and BLE
- GPIO Pins: Multiple programmable I/O pins

Purpose

- Controls the entire system.
- Processes sensor data.
- Maintains worker count.
- Detects fire conditions.
- Uploads data to ThingSpeak.
- Displays information on the LCD.

Flame Sensor Module

Quantity: 1

The flame sensor is used to detect the presence of fire by sensing infrared radiation emitted from flames. When a flame is detected, it sends a digital signal to the ESP32, which activates the emergency alarm and updates the cloud platform.

Specifications

- Operating Voltage: 3.3–5 V
- Digital Output
- Detection Angle: Approximately 60°
- Adjustable Sensitivity

Purpose

- Detect fire.
- Trigger emergency alarm.
- Initiate evacuation process.

Infrared (IR) Obstacle Sensors

Quantity: 2

Two infrared obstacle sensors are used to monitor worker movement through the entrance and exit. One sensor is placed at the entry point and the other at the exit point. The ESP32 uses these signals to calculate and maintain the number of workers currently present inside the industrial premises.

Specifications

- Operating Voltage: 5 V
- Digital Output
- Detection Distance: 2–30 cm
- Adjustable Detection Sensitivity

Purpose

- Detect worker entry.
- Detect worker exit.
- Maintain occupancy count.

16×2 LCD Display with I2C Module

Quantity: 1

The 16×2 LCD is used to display system information such as worker count, Wi-Fi connection status, and fire alerts. The I2C communication module reduces the number of wiring connections required between the ESP32 and the display.

Specifications

- Display Size: 16 Characters × 2 Lines
- Operating Voltage: 5 V
- Communication Interface: I2C
- I2C Address: 0x27

Purpose

- Display worker count.
- Display system status.
- Display fire alerts.
- Display Wi-Fi connection status.

Active Buzzer

Quantity: 1

The active buzzer is used as an emergency warning device. When the flame sensor detects fire, the ESP32 activates the buzzer to alert workers and initiate evacuation.

Specifications

- Operating Voltage: 3.3–5 V
- Active Type
- High Sound Output

Purpose

- Generate emergency alarm.
- Alert workers during fire incidents.

Breadboard**Quantity: 1**

The breadboard is used to assemble and test the prototype circuit without soldering. It allows easy modification of connections during development and debugging.

Purpose

- Prototype development.
- Temporary circuit assembly.
- Easy testing and troubleshooting.

Jumper Wires**Quantity: As Required**

Jumper wires provide electrical connections between the ESP32 and peripheral devices such as sensors, LCD, and buzzer.

Purpose

- Connect hardware components.
- Ensure reliable electrical connections.

USB Type-C Cable**Quantity: 1**

The USB Type-C cable is used to power the ESP32 board and upload the Arduino program from the computer.

Purpose

- Supply power to the ESP32.
- Upload program code.

3.2 Software Requirements

Arduino IDE

Arduino IDE is the software environment used to write, edit, compile, and upload the program to the ESP32 Development Board. It also provides a Serial Monitor for debugging and testing the system.

Purpose

- Program development.
- Code compilation.
- Upload code to ESP32.
- Serial communication and debugging.

Thing Speak Cloud Platform

ThingSpeak is an Internet of Things (IoT) cloud platform used to store, visualize, and monitor real-time data transmitted by the ESP32 through Wi-Fi. The worker count and fire detection status are uploaded periodically, allowing remote monitoring through graphs and dashboards.

Purpose

- Cloud-based data storage.
- Real-time monitoring.
- Data visualization.
- Remote access to system status.

3.3 Arduino Libraries Used

Wi-Fi Library

The Wi-Fi library enables the ESP32 to connect to a wireless network and communicate with the Thing Speak cloud platform.

Purpose

- Wi-Fi connectivity.
- Internet communication.

Thing Speak Library

The ThingSpeak library provides functions to send worker count and fire status data from the ESP32 to the ThingSpeak IoT cloud.

Purpose

- Upload sensor data.
- Cloud communication.

Wire Library

The Wire library implements I2C communication between the ESP32 and the LCD display.

Purpose

- I2C communication.
- Device interfacing.

LiquidCrystal_I2C Library

The LiquidCrystal_I2C library controls the 16×2 LCD through the I2C interface, allowing messages such as worker count, fire alerts, and system status to be displayed.

Purpose

- LCD control.
 - Display information
-

4. Code for the Solution

4.1 Software Development

The proposed Industrial Fire Safety System was developed using the **Arduino IDE** and programmed in **Embedded C/C++**. The ESP32 microcontroller serves as the central controller, continuously monitoring the flame sensor and infrared sensors while communicating with the ThingSpeak IoT platform through its built-in Wi-Fi module.

The program initializes all peripherals, establishes a Wi-Fi connection, reads sensor values, updates the LCD display, controls the buzzer during emergencies, and periodically uploads worker count and fire status to the ThingSpeak cloud.

The software follows a continuous monitoring approach where sensor data is acquired, processed, displayed locally, and transmitted to the cloud in real time.

4.2 Program Flow

The software executes the following sequence:

1. Initialize ESP32 peripherals.
2. Configure GPIO pins.
3. Initialize LCD display.
4. Connect to the Wi-Fi network.
5. Establish communication with ThingSpeak.
6. Read the flame sensor status.
7. Read the IR sensors to detect worker movement.
8. Update worker count.
9. Display current worker count and system status on the LCD.
10. If fire is detected:
 - Activate the buzzer.
 - Display "FIRE ALERT" on the LCD.
 - Upload fire status to ThingSpeak.
11. Continue monitoring continuously.

4.3 Main Program

```
#include <WiFi.h>
#include <ThingSpeak.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,16,2);

void setup()
{
    Serial.begin(115200);

    lcd.init();
    lcd.backlight();

    pinMode(ENTRY_IR,INPUT);
    pinMode(EXIT_IR,INPUT);
    pinMode(FLAME_SENSOR,INPUT);
    pinMode(BUZZER,OUTPUT);

    WiFi.begin(ssid,password);

    ThingSpeak.begin(client);
}
```

4.4 Reading Sensor Data

The ESP32 continuously monitors both infrared sensors for worker movement and the flame sensor for fire detection.

```
int flame=digitalRead(FLAME_SENSOR);
int entry=digitalRead(ENTRY_IR);
int exit=digitalRead(EXIT_IR);
```

4.5 Worker Counting Logic

Whenever a worker enters through the entrance sensor, the worker count is increased. Similarly, the count decreases when a worker exits through the exit sensor.

```
if(entryDetected)
{
    peopleCount++;
}

if(exitDetected)
{
    if(peopleCount>0)
        peopleCount--;
}
```

4.6 Fire Detection Logic

The flame sensor continuously checks for the presence of fire. If a flame is detected, the ESP32 immediately activates the buzzer and displays an emergency alert.

```
if(flame==LOW)
{
    digitalWrite(BUZZER,HIGH);

    lcd.clear();
    lcd.print("FIRE ALERT");
}
else
{
    digitalWrite(BUZZER,LOW);
}
```

4.7 LCD Display

The LCD continuously displays the worker count and the current safety status.

```
lcd.setCursor(0,0);  
lcd.print("Workers:");  
lcd.print(peopleCount);
```

```
lcd.setCursor(0,1);  
lcd.print("Status: SAFE");
```

4.8 Uploading Data to ThingSpeak

The ESP32 uploads worker count and fire status to the ThingSpeak cloud every few seconds.

```
ThingSpeak.setField(1,peopleCount);  
ThingSpeak.setField(2,fireStatus);
```

```
ThingSpeak.writeFields(channelNumber,apiKey);
```

4.9 Continuous Monitoring Loop

The system continuously repeats all monitoring operations.

```
void loop()  
{  
  readSensors();  
  
  updateWorkerCount();  
  
  detectFire();  
  
  displayStatus();  
  
  uploadThingSpeak();  
}
```

Code Summary

The software developed for the Industrial Fire Safety System performs the following functions:

- Initializes the ESP32 and peripheral devices.
- Establishes Wi-Fi communication with the ThingSpeak cloud platform.
- Continuously monitors the flame sensor for fire detection.
- Detects worker entry and exit using infrared sensors.
- Maintains the current worker count inside the industrial premises.
- Displays worker count and system status on the LCD.
- Activates the buzzer during fire emergencies.

- Uploads real-time worker count and fire status to the ThingSpeak cloud.
- Repeats all monitoring operations continuously to ensure uninterrupted industrial safety monitoring.